

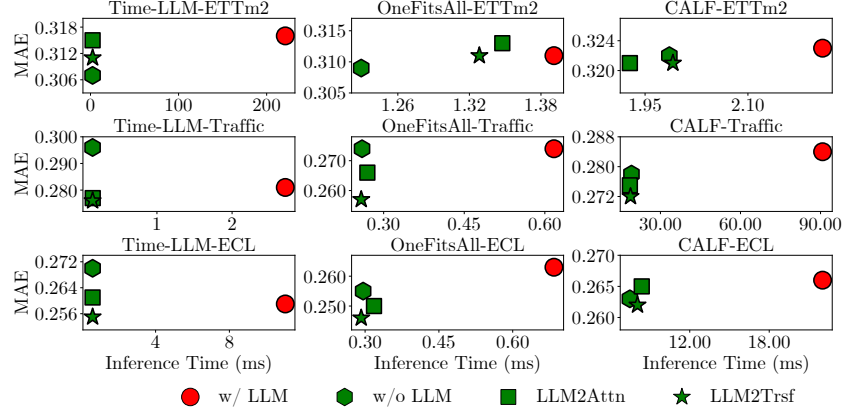


Figure 3: Ablation methods consume less time for inference while providing better forecasting performance. The figure above shows the inference time and prediction accuracy of Time-LLM, OneFitsAll, and CALF on ETTm2, Traffic, and Electricity datasets, averaged across prediction lengths. For more datasets and MSE metrics refer to Figure 7 and Figure 8 in the Appendix.

	Method	Time-LLM (LLaMA)		OneFitsAll (GPT-2)		CALF (GPT-2)	
		# Param (M)	Time (min)	# Param (M)	Time (min)	# Param (M)	Time (min)
ETTh1	w/ LLM	6652	181	85	7.36	180	3.28
	w/o LLM	0.198	0.99	3	0.27	8	0.35
	LLM2Attn	0.202	1.41	5	0.70	10	0.37
	LLM2Trsf	0.336	0.84	8	0.64	13	0.40
Weather	w/ LLM	6642	3003	86	152	180	12
	w/o LLM	0.198	1.91	4	16	8	2.32
	LLM2Attn	0.202	2.22	7	21	10	2.14
	LLM2Trsf	0.336	2.38	10	24	13	1.89

Table 4: In time series tasks, LLM (LLaMA and GPT-2) significantly increases training time. The table shows the number of model parameters (in millions) and total training time (in minutes) for three methods predicting over a length of 96 on ETTh1 and Weather data. Compared with original method “w/ LLM” are “w/o LLM”, “LLM2Attn” and “LLM2Trsf”.

2.3, and 1.2 times longer than the modified models. Examples can be seen in Figure 3, where the green marks (ablation methods) are typically below the red one (LLM) and are positioned towards the left of the axis, indicating a lower computational costs and better forecasting performance. Other datasets and MSE metric refer to Figure 7 and Figure 8 in Appendix. In conclusion, the computational intensity of LLMs in time series forecasting tasks does not result in a corresponding performance improvement.

4.3 Does language model pretraining help performance on forecasting tasks? (RQ3)

Our evaluation in this section indicates that **pretraining with language datasets is unnecessary for time series forecasting**. To test whether the knowledge learned during pretraining meaningfully improves forecasting performance we experimented with different combinations of pretraining and finetuning CALF’s [21] language model on time series.

- **Pretrain + Finetune (Pre+FT)**. This is the original method, wherein a pretrained language model is finetuned on time series data. In the case of CALF, the base language model is frozen and low rank adapters (LoRA) are learned.
- **Random Initialization + Finetune (woPre+FT)**. Does the textual knowledge from pretraining aid time series forecasting? In this method we randomly initialize the weights of the language model (thereby erasing any effect of pretraining) and train the LLM from scratch.
- **Pretrain + No Finetuning (Pre+woFT)**. How much does finetuning on time series improve prediction performance? For this baseline we again leave the language model frozen and forgo

Methods	Pre+FT (GPT-2)		woPre+FT		Pre+woFT		woPre+woFT	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
ETTh1	0.4312	0.4313	0.4284	0.4362	0.4267	0.4342	0.4365	0.4474
ETTh2	0.3838	0.3510	0.3839	0.3508	0.3830	0.3514	0.3872	0.3554
ETTm1	0.3910	0.3963	0.3933	0.4013	0.3898	0.3954	0.3949	0.4028
ETTm2	0.3230	0.2831	0.3221	0.2852	0.3221	0.2827	0.3224	0.2829
Illness	0.8691	1.6996	0.8523	1.6146	0.8742	1.6640	0.8663	1.6381
Weather	0.2737	0.2510	0.2760	0.2520	0.2771	0.2535	0.2776	0.2582
Traffic	0.2844	0.4438	0.2771	0.4409	0.2820	0.4446	0.2863	0.4483
Electricity	0.2660	0.1758	0.2597	0.1669	0.2635	0.1730	0.2663	0.1784
# Wins:	3		8		5		0	

Table 5: Randomly initializing LLM parameters and training from scratch (woPre) achieved better results than using a pretrained (Pre) model. “woFT” and “FT” refer to whether the LLM parameters are frozen or trainable.

Dataset	ETTh1				Illness			
	Sf-all.	Sf-half.	Ex-half	Masking	Sf-all.	Sf-half.	Ex-half	Masking
Time-LLM	51.8%	5.6%	79.6%	32.5%	99.0%	33.6%	34.9%	64.6%
w/o LLM	56.0%	4.5%	89.7%	39.5%	76.5%	20.9%	18.4%	53.0%
LLM2Attn	53.8%	3.3%	92.2%	33.8%	72.7%	20.4%	13.1%	44.6%
LLM2Trsf	50.3%	3.4%	89.2%	34.8%	74.5%	23.0%	14.3%	49.3%
OneFitsAll	62.1%	6.1%	16.6%	31.3%	86.2%	30.9%	36.7%	77.5%
w/o LLM	58.6%	6.1%	19.2%	36.1%	68.9%	13.0%	17.3%	43.5%
LLM2Attn	68.5%	9.0%	15.0%	34.4%	108.3%	39.8%	44.2%	74.2%
LLM2Trsf	58.0%	7.8%	12.6%	30.2%	90.8%	27.4%	40.3%	60.6%
CALF	50.5%	9.6%	5.6%	8.5%	113.0%	47.4%	24.4%	22.9%
w/o LLM	56.2%	12.1%	6.1%	10.4%	118.0%	50.4%	45.8%	28.9%
LLM2Attn	51.9%	10.8%	5.8%	7.3%	87.3%	42.4%	35.1%	25.8%
LLM2Trsf	50.3%	8.5%	5.5%	7.0%	102.6%	56.2%	32.6%	26.0%

Table 6: For the input shuffling/masking experiments on ETTh1 (predict length is 96) and Illness (predict length is 24), the impact of shuffling the input on the degradation of time series forecasting performance does not change significantly before and after model modifications. Results of other predict lengths refer to table 21 in Appendix.

learning LoRAs. Results from this model are therefore indicative of the base language model’s performance without additional guidance on processing time series.

- **Random Initialization + No Finetuning (woPre+woFT).** This baseline is effectively a random projection from the input time series to a forecasting prediction and serves as a baseline comparison with the other methods.

Overall, as shown in Table 5, across 8 datasets using MAE and MSE metrics, the "Pretraining + Finetune" method performed the best 3 times, while "Random Initialization + Finetune" achieved this 8 times. This indicates that language knowledge offers very limited help for forecasting. However, "Pretrain + No Finetuning" and the baseline "Random Initialization + No Finetuning" performed the best 5 times and 0 times, respectively, suggesting that Language knowledge does not contribute meaningfully during the finetuning process. Detailed results refer to Table 20 in Appendix.

In summary, textual knowledge from pretraining provides very limited aids for time series forecasting.

4.4 Do LLMs represent sequential dependencies in time series? (RQ4)

Most time series forecasting methods that use LLMs finetune the positional encoding to help understand the position of time steps in the sequence [4, 49, 21, 5, 31]. We would expect a time series model with good positional representations to show a significant drop in predictive performance when the input is shuffled [45]. We applied three types of shuffling to the time series: shuffling the entire sequence randomly ("sf-all"), shuffling only the first half of the sequence ("sf-half"), and swapping the first and second halves of the sequence ("ex-half"). As shown in Table 6, **LLM-based methods were no more vulnerable to input shuffling than their ablations.** This implies that LLMs do not have unique capabilities for representing sequential dependencies in time series.

Model	LLaMA		w/o LLM		LLM2Attn		LLM2Trsf	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
ETTh1	0.522	0.555	0.543	0.621	0.547	0.611	0.566	0.656
ETTh2	0.394	0.371	0.432	0.407	0.437	0.416	0.433	0.412
ETTm1	0.426	0.404	0.403	0.393	0.428	0.440	0.438	0.457
ETTm2	0.323	0.277	0.319	0.269	0.355	0.316	0.351	0.311
Weather	0.273	0.234	0.271	0.241	0.275	0.241	0.271	0.239
Traffic	0.306	0.429	0.303	0.431	0.302	0.432	0.298	0.432
Electricity	0.270	0.175	0.275	0.175	0.273	0.179	0.274	0.181
# Wins:	8		7		0		1	

Table 7: In few-shot scenarios (10% dataset), LLaMA (Time-LLM) performs similarly to the ablation methods. LLaMA and “w/o LLM” each outperformed the other 8 times. Note that the results of Time-LLM is from the original paper [14].

Model	GPT-2		w/o LLM		LLM2Attn		LLM2Trsf	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
ETTh1	0.543	0.649	0.542	0.642	0.545	0.646	0.544	0.643
ETTh2	0.433	0.434	0.429	0.427	0.433	0.431	0.432	0.434
ETTm1	0.500	0.574	0.499	0.572	0.503	0.581	0.507	0.586
ETTm2	0.339	0.304	0.338	0.303	0.340	0.305	0.340	0.305
Weather	0.286	0.263	0.287	0.265	0.293	0.270	0.286	0.264
Traffic	0.369	0.571	0.337	0.517	0.341	0.518	0.333	0.510
Electricity	0.301	0.220	0.287	0.205	0.292	0.208	0.290	0.206
# Wins:	2		10		0		2	

Table 8: In few-shot scenarios (10% dataset), Ablation methods perform much better than GPT-2 (CALF). Without LLMs, 12 out of 14 cases showed better performance.

4.5 Do LLMs help with few-shot learning in forecasting? (RQ5)

In this section, our evaluation demonstrates that LLMs are still **not meaningfully useful in few-shot learning scenarios**.

While our results indicate that LLMs are not useful for time series forecasting, it is nonetheless possible that knowledge encoded in pretrained weights could help performance in few-shot settings where data are scarce. To evaluate whether this is the case we trained models and their ablations on 10% of each dataset. Specifically, we evaluated LLaMA in Time-LLM methods. The results for LLaMA, shown in Table 7, compared LLaMA with completely removing the LLM (**w/o LLM**). There was no difference, with each performing better in 8 cases. We conducted similar experiments with CALF, a GPT-2-based method. Our results in Table 8 indicate that our ablations can perform better than LLMs in few-shot scenarios.

4.6 Where does the performance come from? (RQ6)

In this section, we evaluate common encoding techniques used in LLM time series models. We find that combining **patching with one-layer attention is a simple and effective choice**.

In subsection 3.2, we found that simple ablations of LLM-based methods did not decrease performance. To understand why such simple methods work so well we selected some popular techniques used for encoding in LLM time series tasks, such as patching [49, 14, 5, 31, 21], decomposition [4, 27]. A basic transformer block also can be used to aid in encoding [21].

The specific results, shown in Table 18 in the Appendix, indicate that a structure combining patching and attention, named “PAttn”, performs better than most other encoding methods on small datasets (with time stamps less than 1 million) and is even comparable to LLM methods. Its detailed structure, as shown in Figure 4, involves applying “instance norm” to the time series, followed by patching and projection. Then, one-layer

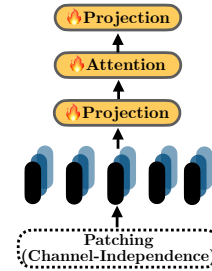


Figure 4: PAttn Model.